



ESCUELA DE
INGENIERÍA EN CIENCIAS Y SISTEMAS
FACULTAD DE INGENIERÍA
UNIVERSIDAD DE SAN CARLOS DE GUATEMALA



Fecha:

00/00/2026

Análisis y Diseño de Sistemas 1

Escuela de Ingeniería de Ciencias Y Sistemas

Tutor: Nombre del tutor

UNIDAD 3:

Garantía de Calidad del

Software

Anuncios Importantes



Anuncio 1



Anuncio 2



Anuncio 3



Anuncio 4



Anuncio 5

Agenda



TEMA 1



TEMA 2



TEMA 3



TEMA 4



TEMA 5

COMPETENCIA(S) QUE DESARROLLAREMOS



1. El estudiante Implementa pruebas unitarias JUnit, pytest, Jest para validar componentes de software
2. El estudiante Aplica principios SOLID SOLID para mejorar calidad del código



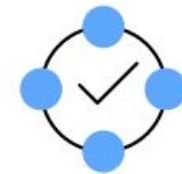
VALOR DE LA SEMANA: Responsabilidad

Las pruebas unitarias y la cobertura de código son herramientas de responsabilidad profesional: garantizan que el código funciona como se espera y que futuras modificaciones no rompen funcionalidad existente.

Aplicar SOLID es una forma de responsabilidad técnica, ya que facilita el mantenimiento y la prueba del sistema. Escribir mocks y stubs demuestra el compromiso del desarrollador con validar su código de forma aislada y confiable.

**CUANDO SE DEFINE REQUERIMIENTOS
FUNCIONALES Y NO FUNCIONALES, SE ESTÁ
DESCRIBIENDO QUÉ DEBE HACER EL SISTEMA
Y CÓMO DEBE COMPORTARSE. A PARTIR DE
AHÍ, SE BUSCA QUE EL SISTEMA SEA
MANTENIBLE, ESCALABLE Y CONFIABLE**

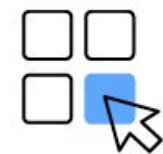




3. End-to-end testing



2. Integration testing

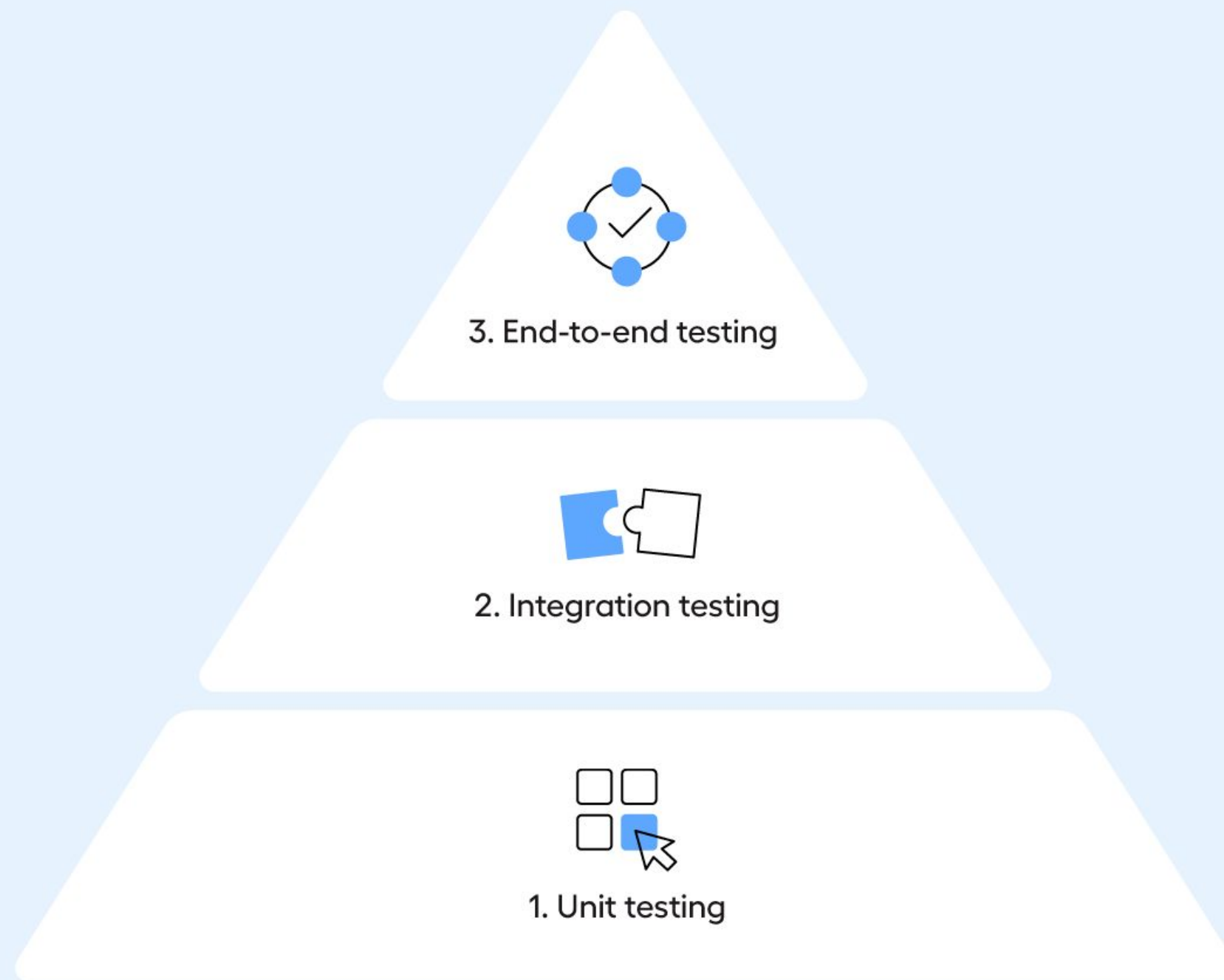


1. Unit testing

Un código que viola SOLID generalmente es difícil de probar — las dependencias están pegadas, las funciones hacen demasiado.

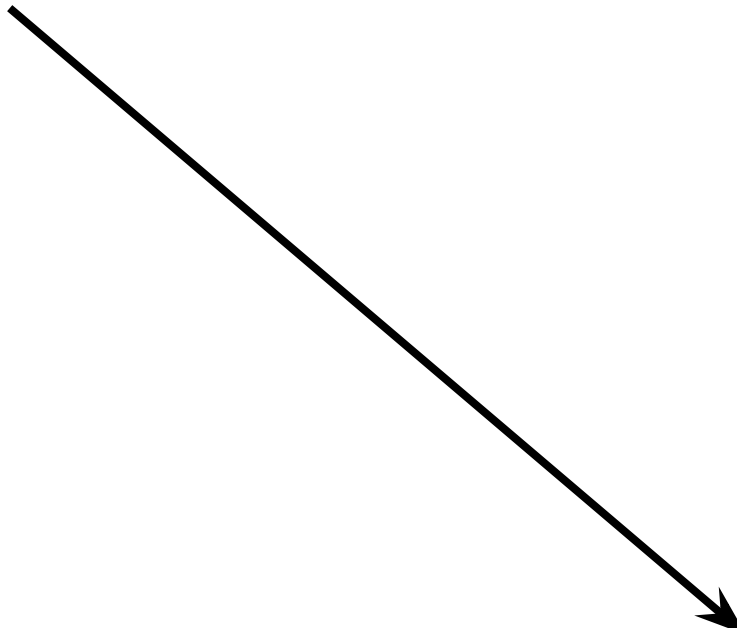
Si tienes dificultad para escribir pruebas, a veces es señal de que el diseño puede mejorar

La Pirámide de Testing



- E2E — Pocas, lentas, costosas. Simulan un usuario real
- Integración — Prueban módulos juntos. Usan mocks/fakes
- Unitarias — Muchas, rápidas, baratas, aisladas ← hoy nos enfocamos aquí

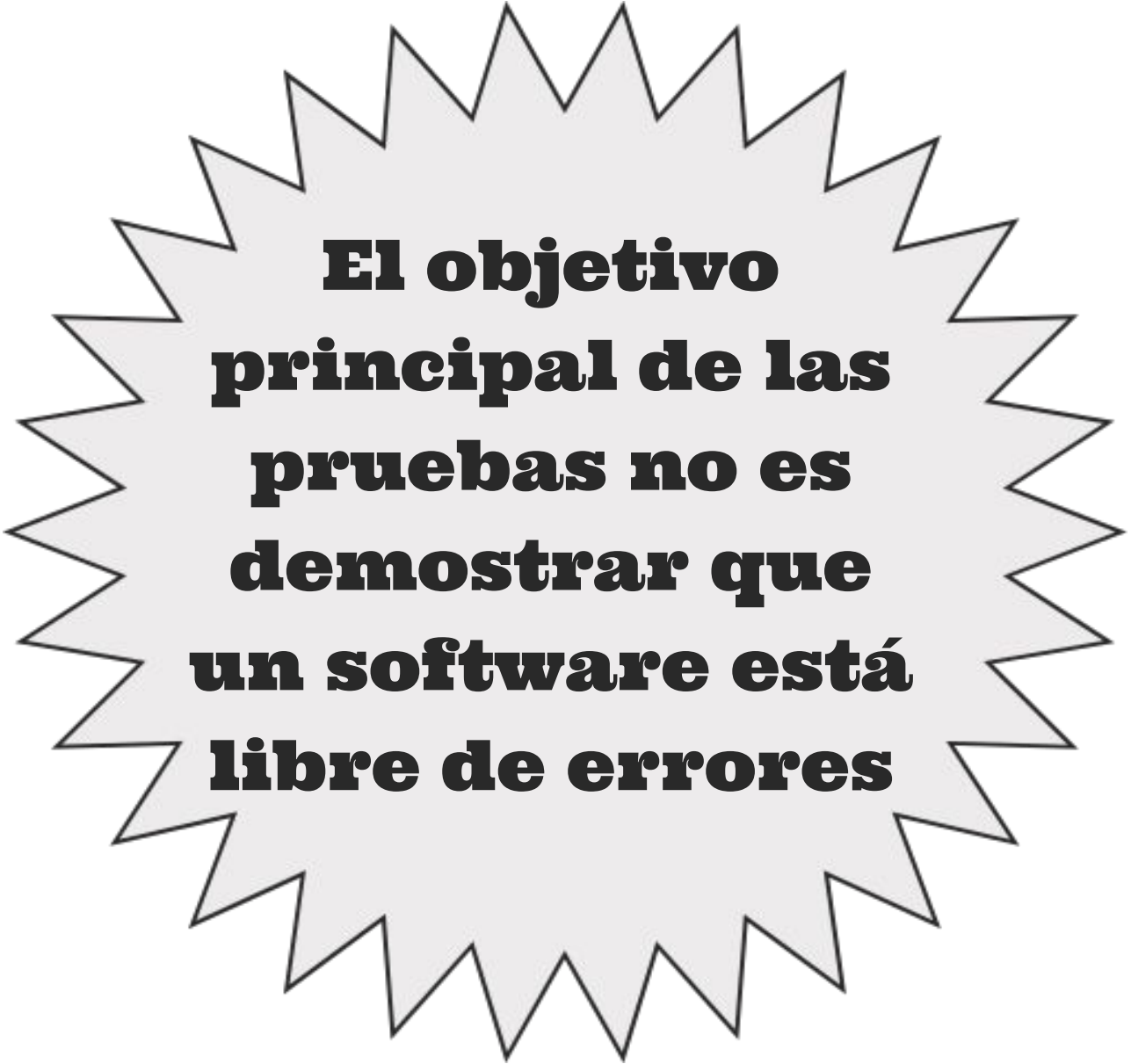
**UNA VEZ QUE EL SISTEMA ESTÁ
CONSTRUIDO SEGÚN REQUERIMIENTOS
CLAROS Y PRINCIPIOS DE DISEÑO, ES
FUNDAMENTAL COMPROBAR QUE
CUMPLE LO ESPERADO. AQUÍ ES
DONDE ENTRAN LAS PRUEBAS DE
SOFTWARE (TESTING).**



- **Primero Diseñamos Bien Con Solid,**
- **Luego escribimos pruebas para verificar que el comportamiento sea el esperado.**

PRUEBAS FUNCIONALES Y NO FUNCIONALES

Las pruebas de software son un proceso sistemático de evaluación de una aplicación para encontrar errores y asegurar que cumple con los requisitos especificados.



El objetivo principal de las pruebas no es demostrar que un software está libre de errores

Se centran en el comportamiento observable del sistema.

Buscan verificar que las funciones implementadas hacen lo que se espera que hagan según los requisitos del negocio.

PRUEBAS FUNCIONALES

Se basan en:

- Casos de uso
- Requisitos funcionales
- Reglas de negocio

Estas pruebas se realizan típicamente con técnicas de caja negra, en las que el evaluador no necesita conocer la lógica interna del código, sino que se enfoca exclusivamente en las entradas y salidas

PRUEBAS FUNCIONALES

PRUEBAS NO FUNCIONALES

Se enfocan en los atributos de calidad del sistema más allá de su funcionalidad básica.

A diferencia de las pruebas funcionales, las pruebas no funcionales a menudo requieren herramientas especializadas y entornos de prueba controlados para simular escenarios específicos.

PRUEBAS NO FUNCIONALES

En lugar de preguntar:

“¿qué hace el software?”



Nos ayudan a responder:

“¿cuán rápido lo hace?”

“¿es seguro?”

“¿es fácil de usar?”

“¿cómo se comporta bajo presión?”

TIPOS DE PRUEBAS FUNCIONALES

EVALÚAN QUÉ HACE EL SISTEMA SEGÚN LOS REQUISITOS DEFINIDOS.

PRUEBA DE UNIDAD (UNIT TEST)

Verifica que una función o componente individual funcione correctamente. Usualmente hecha por desarrolladores.

PRUEBA DE INTEGRACIÓN

Evalúa si múltiples módulos funcionan bien juntos. Asegura que las interfaces entre componentes sean correctas.

PRUEBA E2E

Verifican que todo el sistema complete un flujo de trabajo de principio a fin tal como lo haría un usuario real

PRUEBA DE ACEPTACIÓN (UAT)

Realizada por el cliente o usuario final. Verifica que el software cumple sus expectativas y está listo para producción.

PRUEBA DE REGRESIÓN

Asegura que los cambios o correcciones no hayan roto funcionalidades existentes.

PRUEBA DE HUMO (SMOKE TEST)

Verifica que el sistema arranca y responde a las funciones más esenciales sin fallar de inmediato

TIPOS DE PRUEBAS

FUNCIONALES

IMPLEMENTACIÓN

PRUEBA DE UNIDAD (UNIT TEST)

Se implementa con frameworks como JUnit (Java), pytest (Python), Mocha (JS), etc.

PRUEBA DE INTEGRACIÓN

Se integran módulos y se usan stubs/mocks para probar interacciones

PRUEBA E2E

suelen usar herramientas automatizadas como Selenium, Cypress o Playwright

PRUEBA DE ACEPTACIÓN (UAT)

El cliente o usuario final prueba funcionalidades clave usando casos reales

PRUEBA DE REGRESIÓN

Se reejecutan pruebas antiguas después de cambios. Automatizada con Selenium, Cypress, etc.

PRUEBA DE HUMO (SMOKE TEST)

Se ejecutan pruebas básicas para validar que el sistema inicia y responde

TIPOS DE PRUEBAS NO FUNCIONALES

EVALÚAN CÓMO SE COMPORTA EL SISTEMA BAJO DIFERENTES CONDICIONES.

PRUEBA DE RENDIMIENTO

Mide velocidad, tiempos de respuesta y uso de recursos.

PRUEBA DE CARGA

Evalúa el comportamiento con múltiples usuarios simultáneos dentro de los límites esperados.

PRUEBA DE ESTRÉS

Se lleva el sistema más allá de sus límites para ver cómo se comporta bajo presión extrema.

PRUEBA DE VOLUMEN

Prueba con grandes cantidades de datos para ver cómo afecta al rendimiento.

PRUEBA DE PENETRACIÓN (PENTESTING)

Simula ataques reales para detectar brechas de seguridad. como inyección SQL, XSS y validación de permisos.

TIPOS DE PRUEBAS NO FUNCIONALES

IMPLEMENTACIÓN

PRUEBA DE RENDIMIENTO

Herramientas como JMeter, Gatling, Locust miden tiempos de respuesta

PRUEBA DE CARGA

Se simulan usuarios concurrentes (100, 1,000, etc.) con JMeter, Locust, etc.

PRUEBA DE ESTRÉS

Se fuerza al sistema a sobrecargas intencionales para probar fallos

PRUEBA DE VOLUMEN

Se genera gran cantidad de datos con LoadRunner

PRUEBA DE PENETRACIÓN (PENTESTING)

Simulación de ataques reales, a veces por hackers éticos

PRUEBAS UNITARIAS

Las pruebas unitarias son un tipo de prueba de software en la que se verifica el funcionamiento de una "unidad" de código, como una función, método o clase. Su propósito principal es asegurar que cada parte del código funcione correctamente de manera aislada a pequeñas porciones de código.



MOCKING Y STUBS

El mocking y los stubs son técnicas utilizadas para simular el comportamiento de dependencias externas, como bases de datos o servicios web. Un mock es un objeto que imita el comportamiento de una dependencia, mientras que un stub proporciona respuestas predefinidas.

Término	¿Qué es?	Ejemplo en el proyecto
Mock	Objeto falso que además verifica cómo fue llamado. Espía el flujo.	<code>userRepo.findByEmail</code> en <code>authService.test.js</code>
Stub	Devuelve datos fijos y controlados. Sin lógica propia.	<code>clockStub:</code> <code>{ now: () => fixedNow }</code>
Fake	Implementación ligera pero funcional. Trabaja en memoria.	<code>FakeAppointmentRepo</code> en <code>appointmentService.test.js</code>

Stub miente con datos · Fake trabaja en memoria · Mock espía y verifica

JEST & SUPERTEST

Jest es un framework de pruebas para JavaScript que permite escribir y ejecutar pruebas unitarias y de integración



Supertest es una librería especializada en probar APIs HTTP, permitiendo enviar solicitudes y verificar respuestas de servidores.





Principios SOLID

"El código que viola SOLID es difícil de probar.
El código SOLID casi se prueba solo."



¿Qué son los principios SOLID?

Los principios SOLID son 5 principios fundamentales para el diseño de software orientado a objetos que nos ayudan a:

- ✓ Escribir código más mantenible
- ✓ Facilitar la extensión de funcionalidades
- ✓ Reducir el acoplamiento entre componentes
- ✓ Hacer el código más testeable
- ✓ Prevenir errores al modificar código existente

Nota: Estos principios NO están grabados en piedra. Son guías que se deben aplicar con criterio según el contexto de tu proyecto.

S - Single Responsibility Principle (SRP)

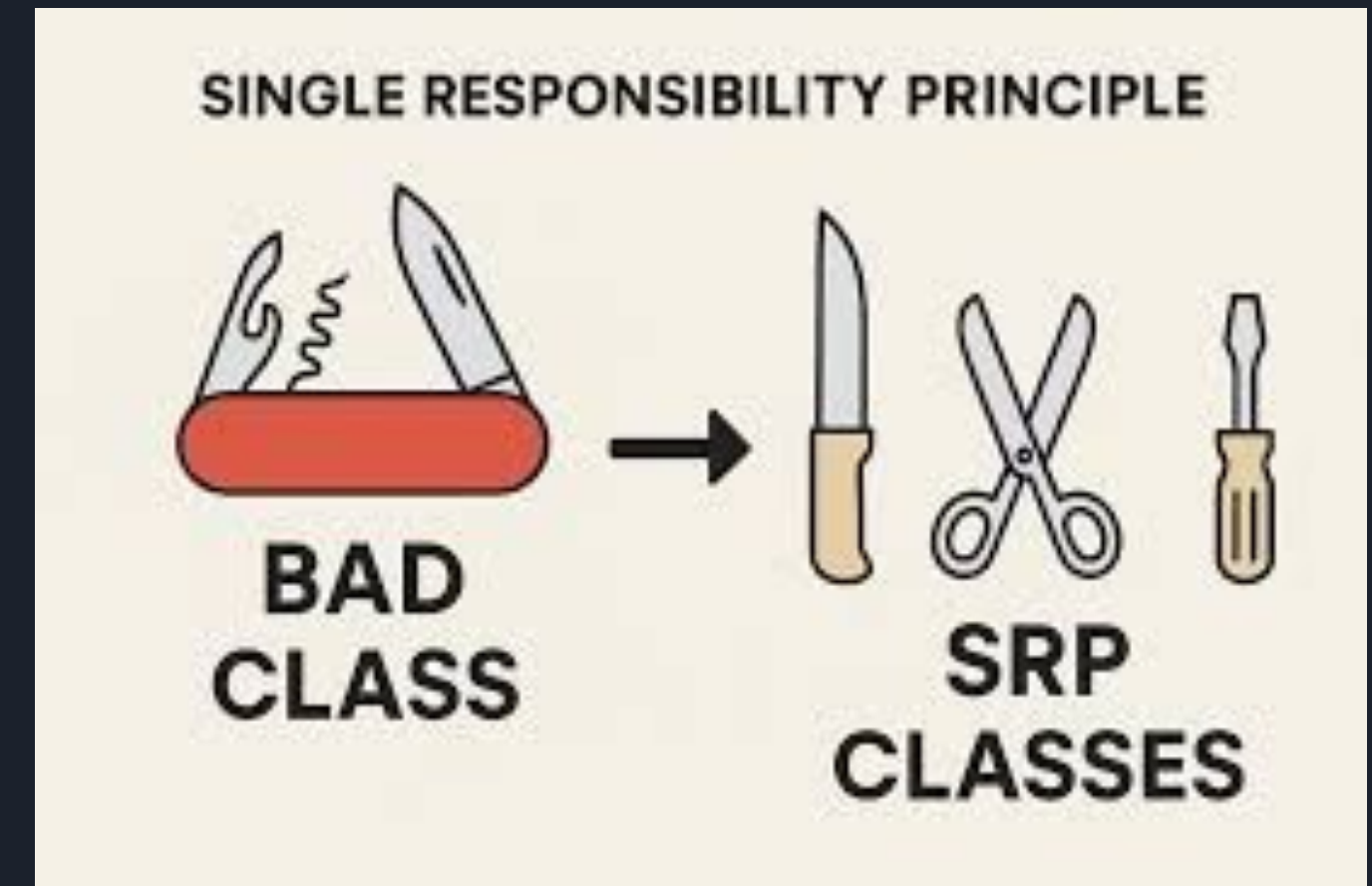
RESPONSABILIDAD ÚNICA

“Una clase debe hacer una cosa y, por lo tanto, debe tener una sola razón para cambiar.”

Esto significa que cada clase debe tener una única responsabilidad o propósito bien definido.

¿Por qué es importante?

- Si una clase hace muchas cosas, un cambio en una funcionalidad puede romper otras
- Es más difícil de entender y mantener
- Dificulta las pruebas unitarias





O: Open/Closed Principle

ABIERTO/CERRADO

“Las clases deben estar abiertas a la extensión y cerradas a la modificación.”

- Abiertas para extensión: Puedes agregar nuevas funcionalidades
- Cerradas para modificación: No debes modificar el código existente que ya funciona

Deberíamos poder agregar nuevas funciones sin tocar el código existente para la clase.

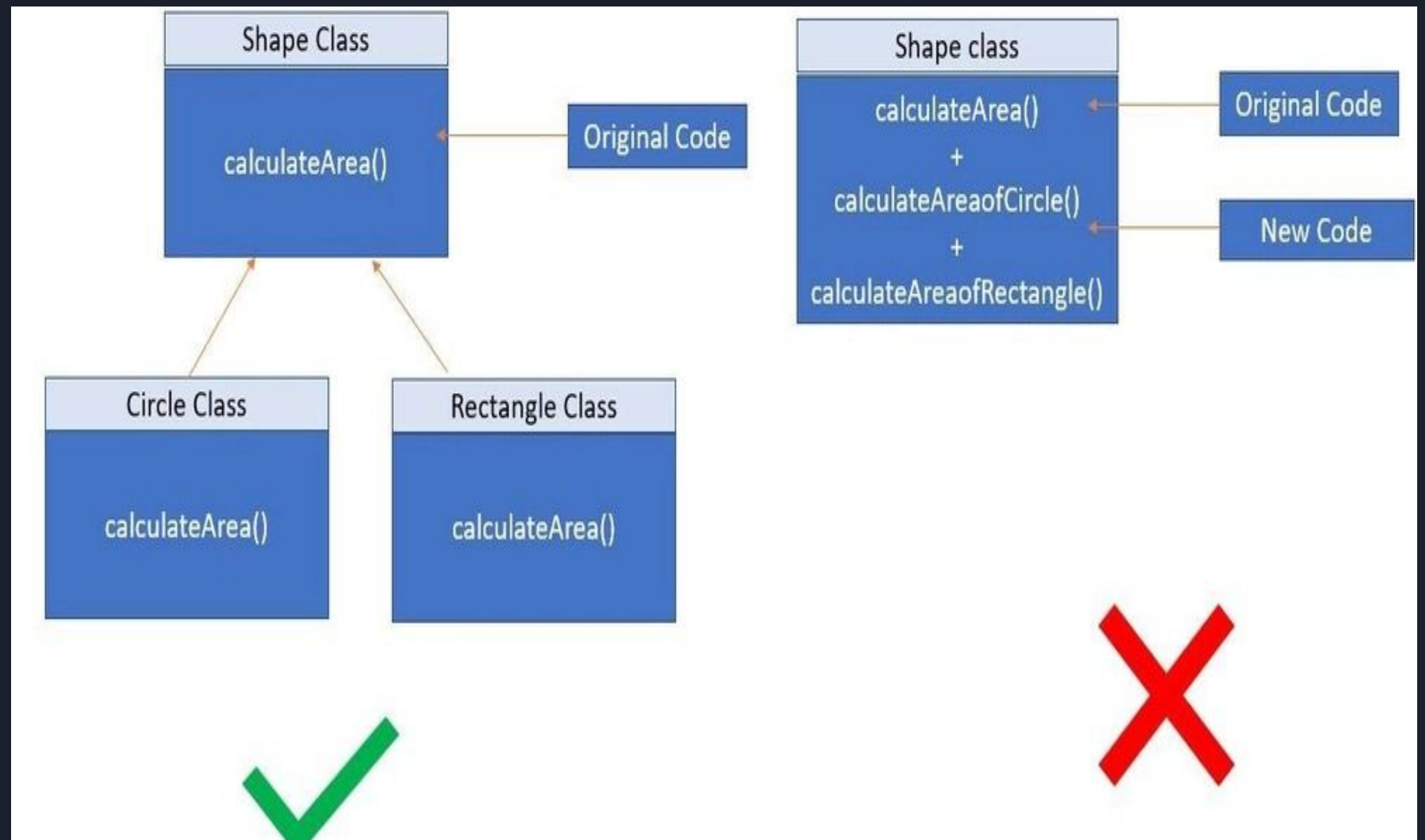
Esto se debe a que cada vez que modificamos el código existente, corremos el riesgo de crear errores potenciales. Por lo tanto, debemos evitar tocar el código de producción probado y confiable (en su mayoría) si es posible.

O: Open/Closed Principle

ABIERTO/CERRADO

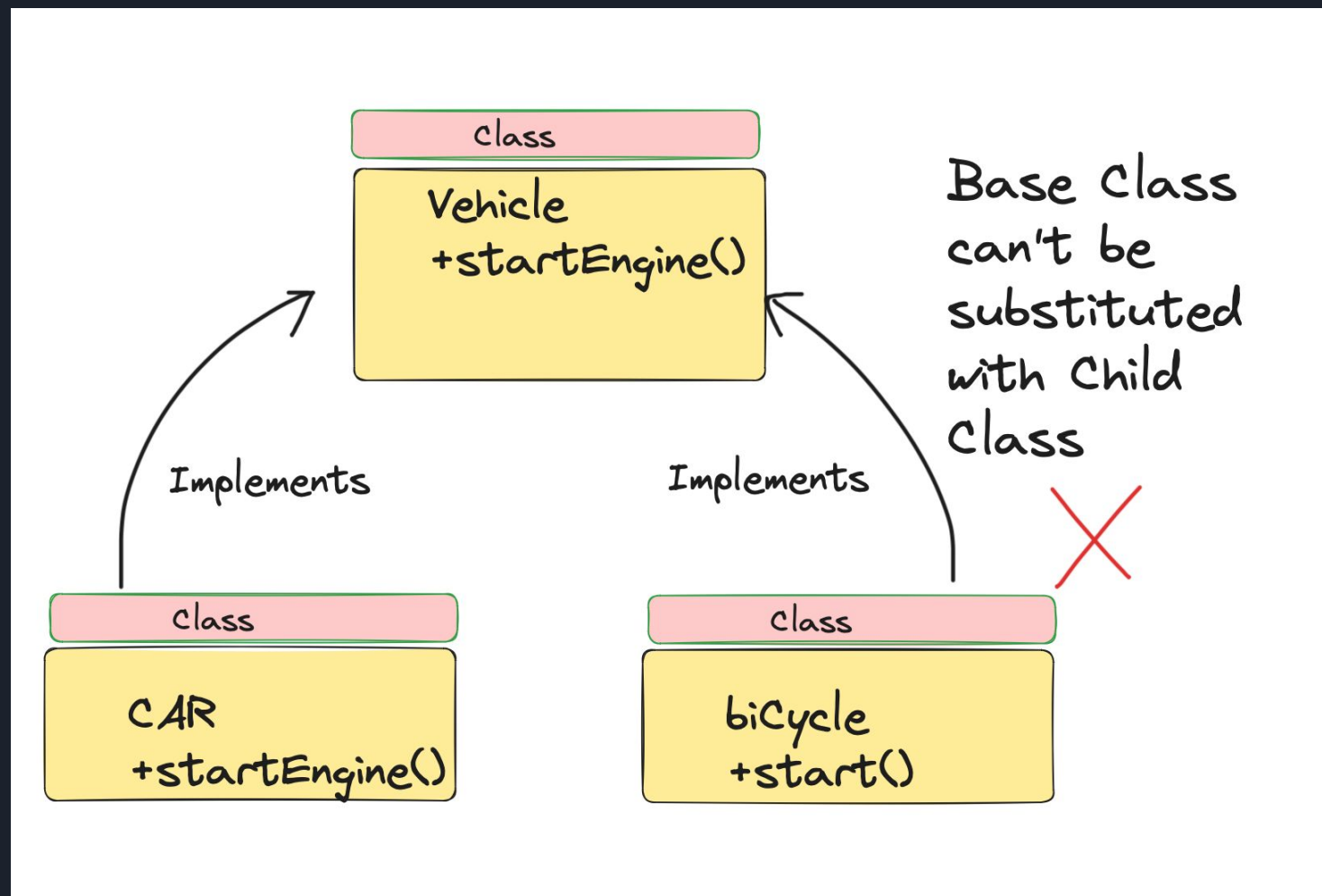
Pero, ¿cómo vamos a agregar una nueva funcionalidad sin tocar la clase?

Por lo general, se hace con la ayuda de interfaces y clases abstractas.



L: Liskov Substitution Principle

SUSTITUCIÓN DE LISKOV



"Las subclases deben ser sustituibles por sus clases base sin alterar el correcto funcionamiento del programa"

Esto significa que, dado que la clase B es una subclase de la clase A, deberíamos poder pasar un objeto de la clase B a cualquier método que espere un objeto de la clase A y el método no debería dar ningún resultado extraño en ese caso.

Este es el comportamiento esperado, porque cuando usamos la herencia asumimos que la clase secundaria hereda todo lo que tiene la superclase. La clase secundaria extiende el comportamiento, pero nunca lo reduce.

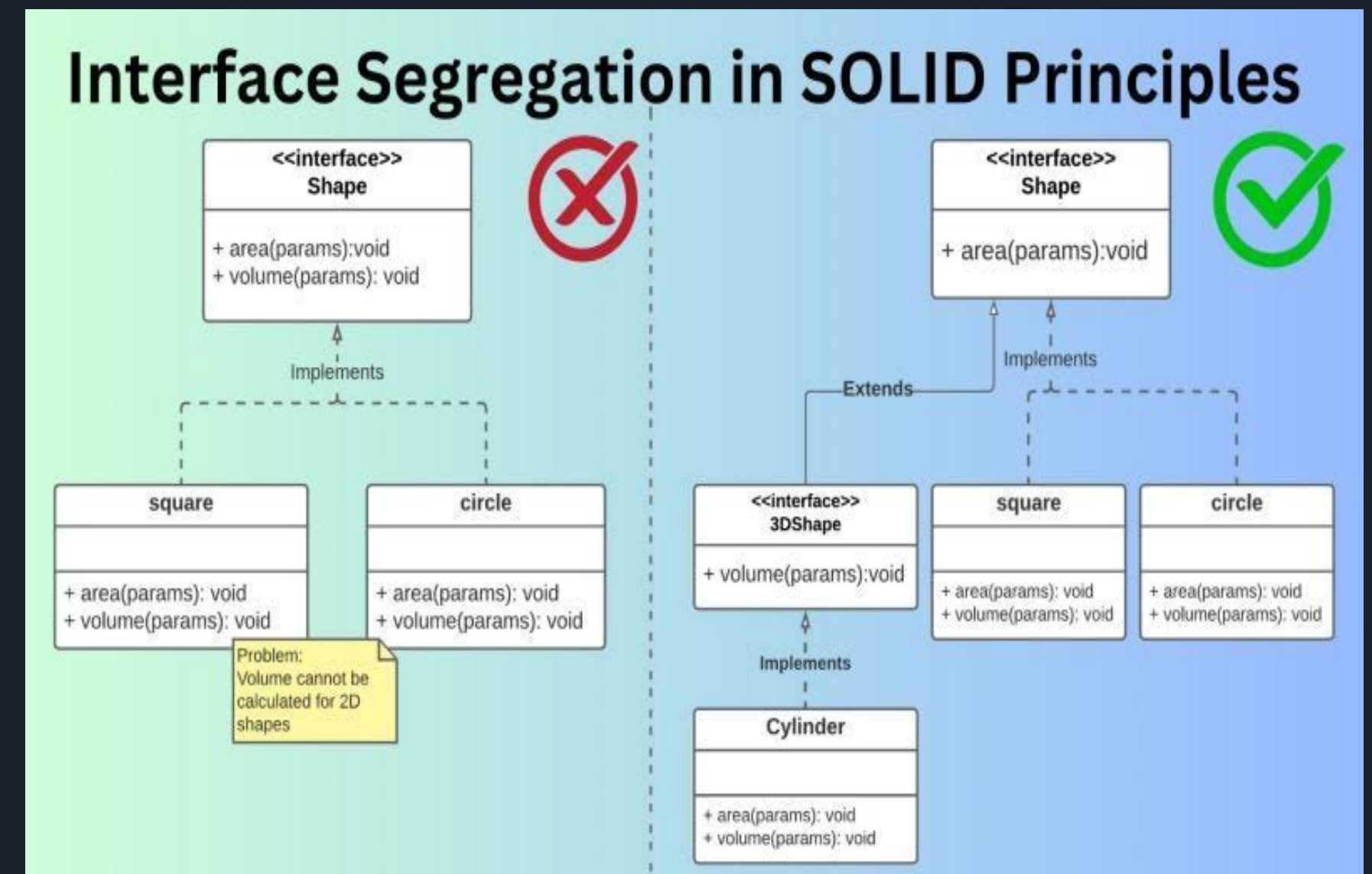
I: Interface Segregation Principle

SEGREGACIÓN DE INTERFACES

“Muchas interfaces específicas del cliente son mejores que una interfaz de propósito general. No se debe obligar a los clientes a implementar una función que no necesitan.”

¿Por qué es importante?

- Se evita forzar a las clases a implementar métodos que no necesitan
- Cada clase implementa solo lo que realmente usa
- Código más limpio y mantenible



D: Dependency Inversion Principle

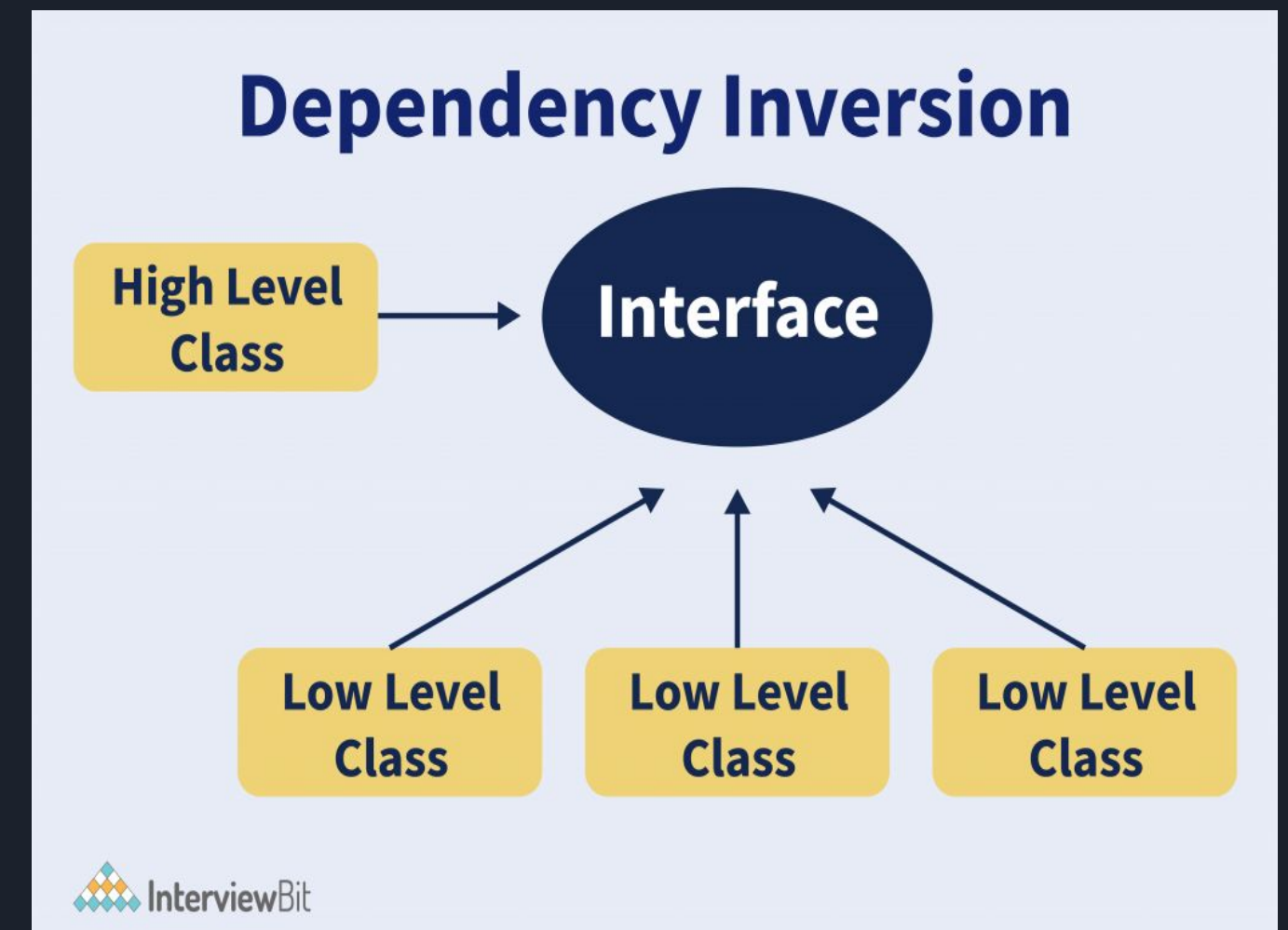
INVERSIÓN DE DEPENDENCIAS

"Los módulos de alto nivel no deben depender de módulos de bajo nivel. Ambos deben depender de abstracciones"

- **Módulos de alto nivel:** Lógica compleja del negocio
- **Módulos de bajo nivel:** Utilidades, detalles de implementación (DB, API, archivos)
- **Abstracción:** Interfaz o clase abstracta

¿Por qué es importante?

- Facilita cambiar la implementación sin afectar la lógica del negocio
- Permite testear con mocks fácilmente
- Reduce el acoplamiento entre componentes

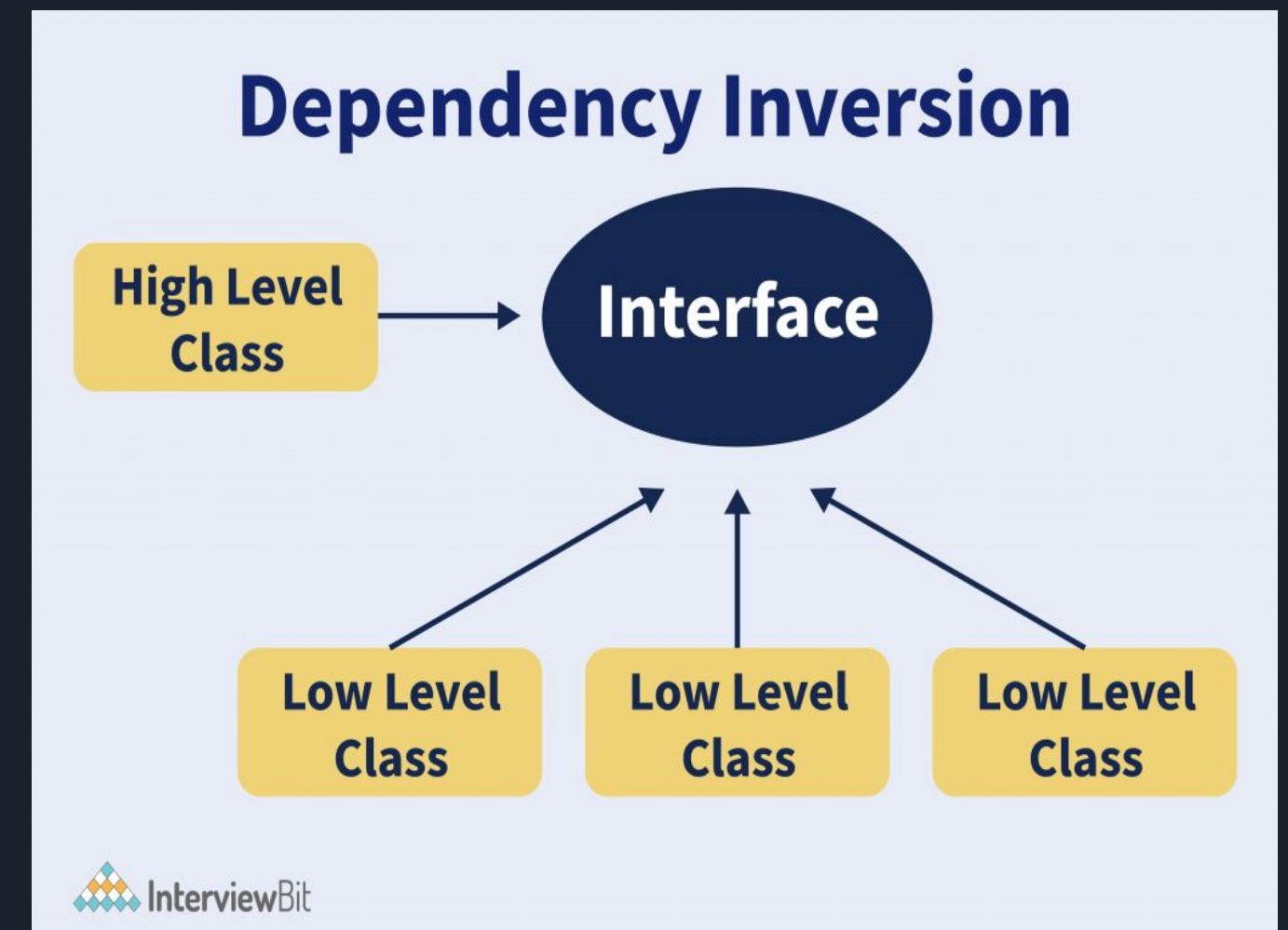


D: Dependency Inversion Principle

INVERSIÓN DE DEPENDENCIAS

La definición de este principio según Robert C. Martin consta de dos partes:

- Los módulos de alto nivel no deben depender de módulos de bajo nivel. Ambos deberían depender de abstracciones.
- Las abstracciones no deben depender de los detalles. Los detalles deben depender de las abstracciones.





RESUMEN — LOS 5 PRINCIPIOS

Letra	Principio	Pregunta Clave
S	Single Responsibility	¿Esta clase tiene una sola razón para cambiar?
O	Open / Closed	¿Puedo agregar comportamiento sin modificar código existente?
L	Liskov Substitution	¿Cualquier subclase puede reemplazar a su clase base sin problemas?
I	Interface Segregation	¿Las interfaces son específicas y no tienen métodos innecesarios?
D	Dependency Inversion	¿Las clases reciben sus dependencias desde afuera (inyección)?



¿Cuándo aplicar SOLID?

SÍ aplicar:

- Proyectos grandes o que crecerán
- Código que se mantendrá por años
- Equipos de varios desarrolladores
- Librerías o frameworks

NO sobre-aplicar:

- Scripts pequeños de una sola vez
- Prototipos rápidos
- Proyectos de aprendizaje muy simples

CONCEPTOS CLAVE APRENDIDOS

- Las pruebas funcionales verifican que el sistema haga lo que debe, basándose en entradas y salidas observables
- Las pruebas no funcionales evalúan atributos como rendimiento, seguridad y comportamiento bajo presión, utilizando herramientas especializadas

CONCEPTOS CLAVE APRENDIDOS

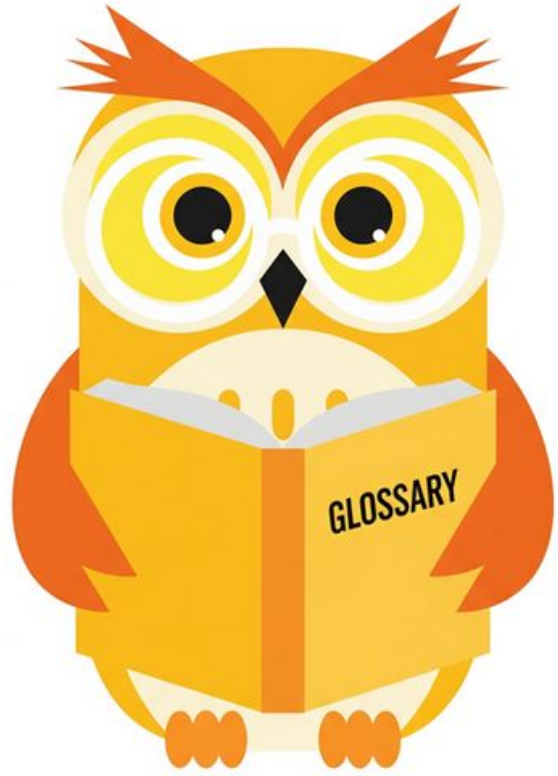
- Las pruebas funcionales verifican que el sistema haga lo que debe, basándose en entradas y salidas observables
- Las pruebas no funcionales evalúan atributos como rendimiento, seguridad y comportamiento bajo presión, utilizando herramientas especializadas

REFERENCIAS

- **Pruebas unitarias y frameworks**
JUnit (Java): <https://junit.org/junit5/>
pytest (Python): <https://docs.pytest.org/>
Jest (JavaScript): <https://jestjs.io/>
Supertest (APIs HTTP en Node.js): <https://github.com/ladjs/supertest>
- **Pirámide de Testing y pruebas automatizadas**
Selenium: <https://www.selenium.dev/>
Cypress: <https://www.cypress.io/>
Playwright: <https://playwright.dev/>
- **Pruebas de rendimiento y carga**
Apache JMeter: <https://jmeter.apache.org/>
Gatling: <https://gatling.io/>
Locust: <https://locust.io/>
LoadRunner: <https://www.microfocus.com/en-us/products/loadrunner-professional/overview> (microfocus.com in Bing)
- **Pruebas de seguridad y pentesting**
OWASP (Open Web Application Security Project): <https://owasp.org/>
- **Principios SOLID (Robert C. Martin)**
Clean Code y SOLID Principles: <https://www.oreilly.com/library/view/clean-code/9780136083238/> (oreilly.com in Bing)



Ejemplo práctico



Practiquemos lo aprendido



Nombre de la actividad:

Cantidad de participantes:

Doy fe que esta actividad está
planificada en dtt (Sí/No):

Hora de inicio:

Hora de fin:

Duración (min):

Participantes: llenar las siguientes cajas de texto (tomar información del chat del meet)

Recursos

- Se colocará el recordatorio de las actividades según corresponda



DUDAS ¿?



**¡GRACIAS POR SU
ATENCIÓN!**

RECUERDA QUE TENEMOS NUESTRO FORO SEMANAL DONDE PUEDES CONSULTAR CUALQUIER DUDA
QUE TE SURJA EN LA SEMANA